

Detecting Phishing Websites: Eight Classifiers, One Clean Hold-Out, and a Re-Audit of a 2022 Result

Furkan Reha Tutaş

September 2026

Abstract

Phishing sites imitate legitimate ones to harvest credentials. The UCI *Phishing Websites* dataset encodes 11,055 sites as 30 hand-crafted features in $\{-1, 0, 1\}$, each a rule of thumb about the URL, the page’s HTML or the domain. This report rebuilds a 2022 course project on that dataset as a tested Python package and re-runs the study with a protocol that keeps a stratified 20,% hold-out untouched until the end. Eight classifiers are compared. The non-linear models land within one point of each other at 0.96–0.98 test accuracy; random forest and default-parameter LightGBM tie for first at 0.977. Two features, the anchor-URL score and the SSL state, carry most of the signal. The re-run also corrects two findings of the 2022 report: its headline LightGBM accuracy of 0.98 does not reproduce (the report’s tuned parameters give 0.958, library defaults give 0.977), and the label column was read with the wrong polarity. Code, data loader, tests and every figure are at github.com/specialone0007/detectingPhishingWebsites.

Contents

1 Problem	2
2 Dataset	2
3 Feature analysis without a model	2
4 Evaluation protocol	4
5 Models	4
6 Results	5
7 What the 2022 report got wrong	7
8 Conclusion	8
9 Reproducing	8
10 References	8

1 Problem

A phishing website is a page built to look like a bank, a mail provider or a shop so that a visitor types in credentials that go to the attacker. Blocklists catch known sites; a classifier can flag *new* ones from properties of the URL and the page itself. The task here is binary: given a feature vector describing one site, decide whether it is phishing.

Two properties of the setting shape the evaluation. First, the two error types are not symmetric: a missed phishing site (false negative) costs a user their password, a false alarm costs a click on “proceed anyway”. Recall on the phishing class therefore matters more than raw accuracy. Second, the features are cheap heuristics that phishers can partly game, so a model that leans on one or two of them is fragile even when it scores well.

2 Dataset

UCI Machine Learning Repository id 327, *Phishing Websites* (Mohammad, Thabtah and McCluskey, 2012–2015). 11,055 sites: 4,898 phishing (44.3,%), 6,157 legitimate (55.7,%). Thirty features, each already discretised by its authors into -1 (“looks like phishing”), 1 (“looks legitimate”) and, for some, 0 (“suspicious”). The features fall into four groups.

Group	Features
Address bar (12)	IP address in URL, URL length, shortening service, @ symbol, // redirect, prefix/suffix -, sub-domains, SSL state, registration length, favicon, non-standard port, https token in domain
Abnormal (6)	request URL, anchor URL, links in <code><script>/<meta>/<link></code> tags, server form handler, submit-to-email, abnormal URL
HTML / JavaScript (5)	redirects, onmouseover status-bar change, right-click disabled, pop-up window, iframe
Domain (7)	domain age, DNS record, web traffic rank, PageRank, Google index, links pointing to page, statistical report

The label column **Result** follows the same convention as the features: -1 is phishing, 1 legitimate. The package maps it to `is_phishing` $\in \{0,1\}$ once, in the loader, so that “positive” means “phishing” everywhere downstream. Class balance is close enough to even that no resampling is needed; the majority-class baseline is 0.557.

3 Feature analysis without a model

3.1 How good is each feature alone?

Because every feature is itself a tiny classifier, we can ask how often its verdict agrees with the label. Encode the label the same way ($y = -1$ phishing, $y = 1$ legitimate) and define the **agreement** of feature A as

$$\text{agr}(A) = P(A = y \mid A \neq 0),$$

the accuracy of the rule “trust A ” on the rows where A does not abstain. This is the *feature validity value* of Zhu et al. (OFS-NN, 2019) written as one number per feature. A value of 0.5 is a coin flip; a value far below 0.5 is informative but inverted.

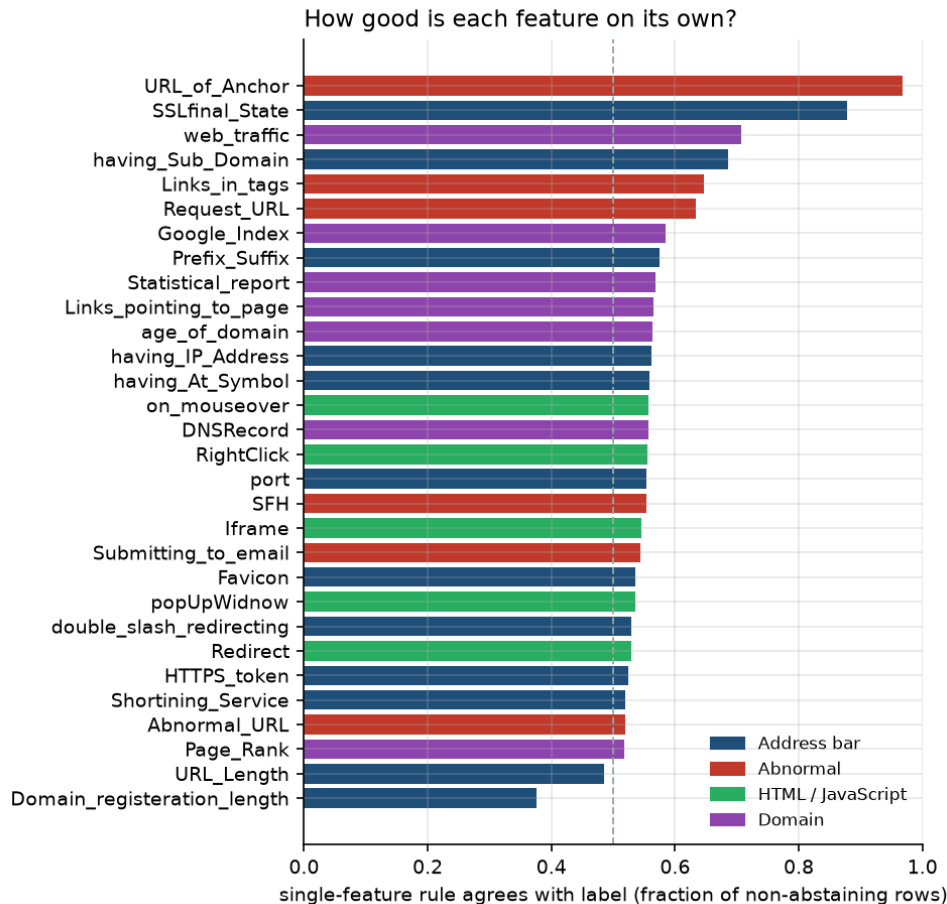


Figure 1: Single-feature agreement with the label, coloured by feature group. Dashed line: coin flip.

Two features stand far apart: `URL_of_Anchor` (0.967) and `SSLfinal_State` (0.878). A third tier (`web_traffic`, `having_Sub_Domain`, `Links_in_tags`, `Request_URL`) sits at 0.63–0.71. Twenty of the thirty features are within 0.1 of a coin flip. `Domain_registration_length` (0.375) is *inverted*: short registrations are, in this sample, more often legitimate than the rule assumes, which is a warning that some of these heuristics were written for the web of 2012.

3.2 Redundancy

Twenty-six feature pairs have $|r| > 0.5$ (Pearson, on the $\{-1, 0, 1\}$ codes). The HTML/JavaScript block is nearly one variable: `Favicon`–`popUpWidnow` $r = 0.94$, `Favicon`–`port` 0.80, `port`–`Submitting_to_email` 0.80, and so on. The address-bar heuristics `Shortning_Service`, `double_slash_redirecting`, `HTTPS_token` and `Abnormal_URL` form a second cluster at $r \approx 0.72$ –0.84. Tree ensembles are indifferent to this; the linear models below pay for it.

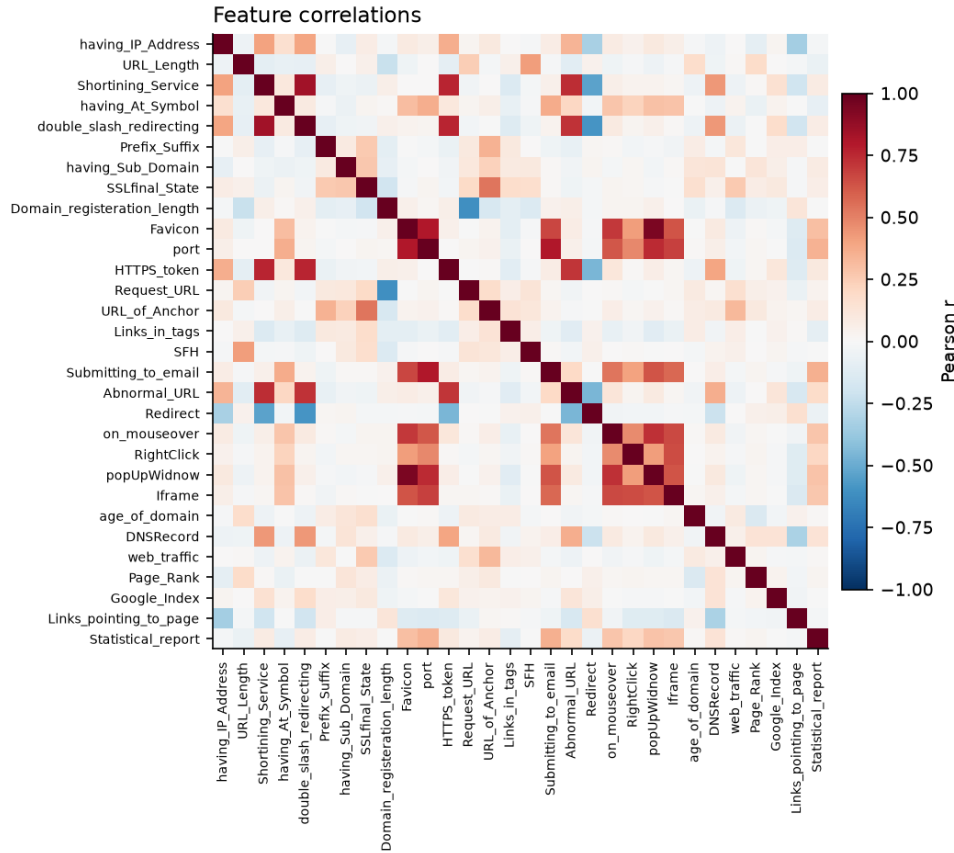


Figure 2: Pearson correlation between the 30 features.

4 Evaluation protocol

The 2022 report tuned hyper-parameters and reported accuracy on what appears to be the same 70/30 split. That inflates the headline number by an unknown amount. The protocol here:

1. **One stratified 80/20 split**, seed 42: 8,844 training rows, 2,211 test rows. The test rows are written to `results/test-index.npy` and touched exactly once, at the end.
2. **5-fold stratified cross-validation on the training rows** for every model, reported as mean $\pm$ standard deviation. This is the number a practitioner would use to choose a model.
3. **One fit on all training rows, one prediction on the test rows**: accuracy, precision, recall and F_1 for the phishing class, and ROC-AUC from predicted probabilities.

Hyper-parameters are those of the 2022 report where it gave them (XGBoost, LightGBM, both SVMs), sensible defaults otherwise. No further tuning was done, deliberately: the point is to see how the published settings hold up on data they were not chosen on.

5 Models

Eight scikit-learn-compatible estimators behind one factory (`make_model(name, seed)`). Scaled inputs for the SVMs, the MLP and logistic regression; raw codes for the trees.

name	model	notes
logistic	logistic regression, $C = 1$	linear baseline
svm_linear	SVM, linear kernel, $C = 0.1$	2022 params
svm_rbf	SVM, RBF kernel, $C = 100$, $\gamma = 0.125$	2022 params
mlp	MLP 64–64–32, $\alpha = 10^{-3}$, early stopping	stand-in for the 2022 NN
xgboost	1000 trees, $\eta = 0.1$, depth 6, $\gamma = 1$, $\lambda = \alpha = 0.15$, 1024 bins	2022 params
lightgbm	1000 trees, $\eta = 0.05$, depth 10, 63 leaves, extra trees, feature fraction 0.9, bagging 0.8 every 8	2022 params
lightgbm_default	500 trees, $\eta = 0.05$, everything else library default	control
random_forest	500 trees, $\sqrt{p}$ features per split	control

6 Results

Table 3: Cross-validated and held-out performance, sorted by test accuracy. Positive class = phishing.

model	CV acc	test acc	prec.	recall	F_1	AUC	fit s
LightGBM (defaults)	$0.966 \pm .002$	0.977	0.982	0.964	0.973	0.996	0.8
Random forest	$0.970 \pm .002$	0.977	0.980	0.966	0.973	0.997	1.1
SVM, RBF	$0.962 \pm .003$	0.975	0.978	0.965	0.972	0.987	19.3
MLP	$0.959 \pm .004$	0.972	0.988	0.948	0.968	0.995	1.6
XGBoost (2022 params)	$0.958 \pm .002$	0.968	0.970	0.958	0.964	0.995	1.2
LightGBM (2022 params)	$0.950 \pm .004$	0.958	0.969	0.935	0.952	0.990	30.8
Logistic regression	$0.928 \pm .004$	0.928	0.934	0.901	0.917	0.979	0.0
SVM, linear	$0.925 \pm .005$	0.925	0.934	0.895	0.914	0.977	2.8

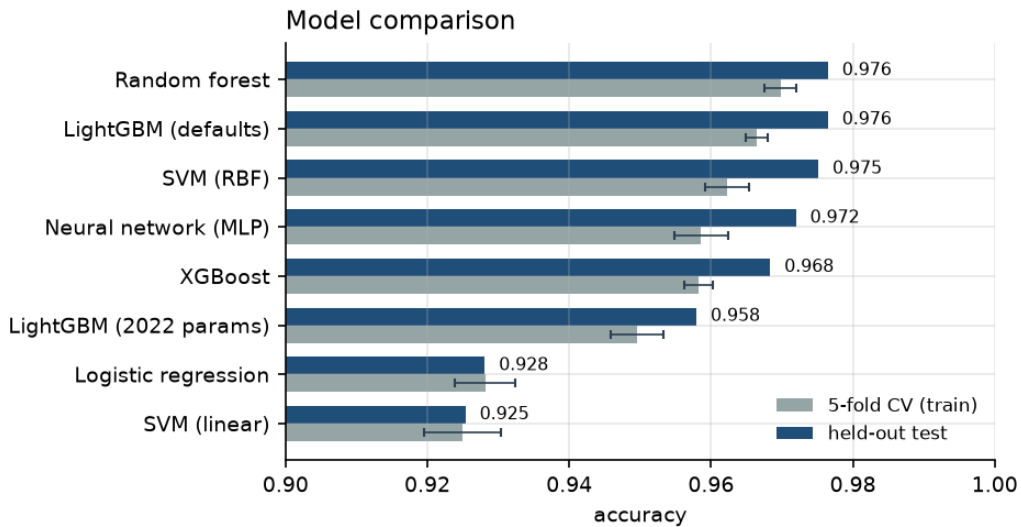
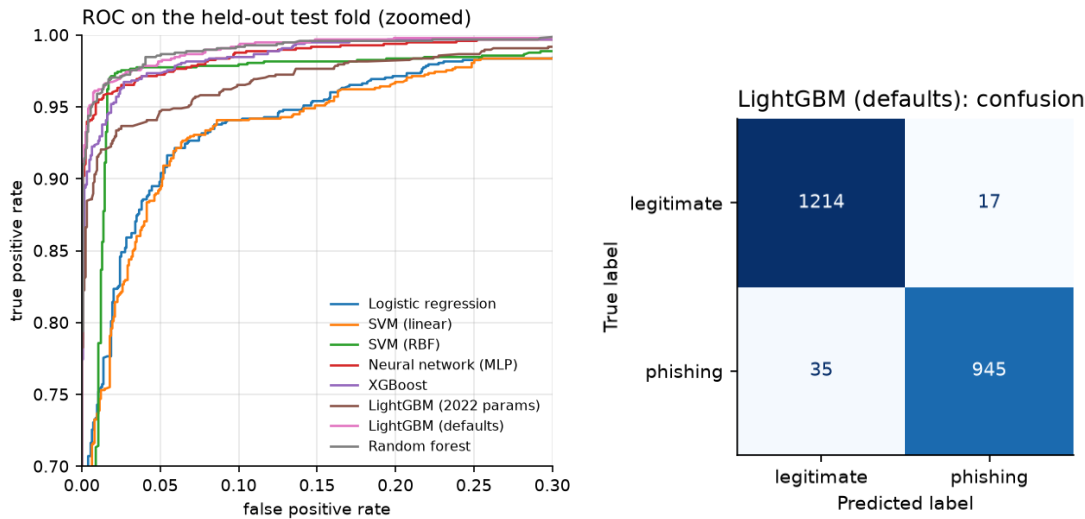


Figure 3: Cross-validated (grey, with ± 1 s.d.) and held-out (blue) accuracy.

Linear versus non-linear. The two linear models stop at 0.925–0.928. Every non-linear model

clears 0.958. The gap is the interactions: the anchor and SSL features are only decisive *together*, and the correlated HTML block adds nothing to a linear model that it did not already have.

Among the non-linear models the spread is one point. Random forest and default LightGBM tie at 0.977 with almost identical confusion matrices (17 false alarms, 35 misses for LightGBM). SVM-RBF is a hair behind at $20\times$ the training time. CV and test agree to within one standard deviation for every model, which is what an honest protocol should produce.



What the trees use. LightGBM's split gain puts 44,% on URL_of_Anchor and 17,% on SSLfinal_State; the next four features share another 14,%. The ranking agrees with the model-free agreement scores of §3, which is reassuring: the ensembles are not finding signal the simple analysis missed, they are combining the signal that is visibly there.

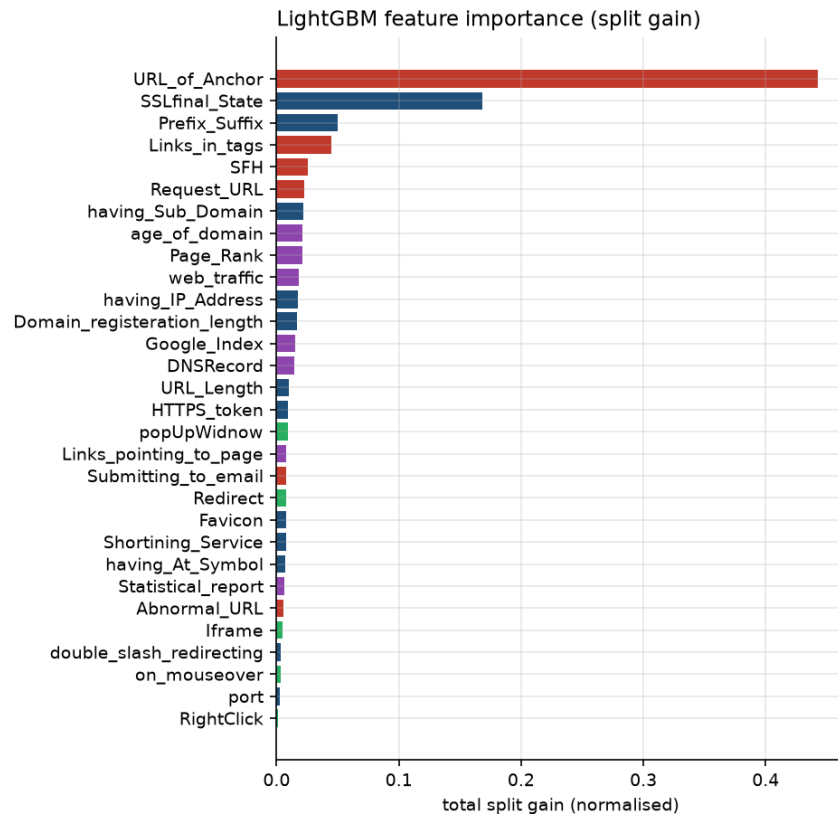


Figure 4: LightGBM feature importance (total split gain, normalised).

The hard core. 22 test sites (1.0,%) are misclassified by all five non-linear models. Twenty of them are phishing sites; nineteen carry a valid SSL state and a merely “suspicious” anchor score, i.e. they look legitimate on exactly the two features the models lean on. That is the fragility flagged in §1: an attacker who buys a certificate and keeps anchors on-domain defeats most of this feature set. Better recall on this residue needs features these 30 do not contain (content, visual similarity, reputation over time), not a better classifier.

7 What the 2022 report got wrong

The 2022 course project reached the same qualitative ranking (tree ensembles and RBF-SVM on top, linear SVM at the bottom) but two of its concrete claims do not survive a clean re-run.

The LightGBM headline. The report gave 0.98 accuracy for LightGBM with the parameters listed in §5. On the held-out fold those parameters give 0.958, the lowest of the non-linear models, and 0.950 in cross-validation. The same library with default parameters gives 0.977. The tuning therefore *hurt*: `extra_trees` with 63 leaves at depth 10 and 1,000 rounds without early stopping over-fits 8,800 rows of ternary features. The most plausible reading of the 0.98 is that it was measured on the split the parameters were selected on. This is the standard failure mode of tuning without a separate validation fold, and it is the reason §4 exists.

Label polarity. The report treats `Result = 1` as phishing. In the UCI documentation and in the

feature-encoding paper the dataset comes from, -1 is phishing and 1 legitimate, matching every feature's convention. Accuracy and ROC-AUC are symmetric under the swap, so the report's headline numbers are unaffected, but its per-class precision and recall describe the wrong class, and its statement that phishing is the majority class (55,%) is backwards. The package resolves this once in the loader.

Neither error changes the engineering conclusion; both change what the numbers *mean*.

8 Conclusion

On the UCI Phishing Websites data, any reasonable non-linear classifier reaches 0.96–0.98 accuracy and 0.95–0.97 phishing recall, and the choice between them is a matter of training cost, not quality. Two features do most of the work. The remaining one per cent of errors are phishing sites that look legitimate on precisely those two features, which no model on these inputs will fix. The practical lesson is about protocol, not models: a hold-out that is chosen once and never tuned on turned a reported 0.98 into a reproducible 0.977 for one model and 0.958 for another.

9 Reproducing

```
pip install -e ".[dev]"
pytest                # 17 tests, synthetic data, ~3 s
phishing-experiments  # downloads the dataset once; ~90 s;
                      # writes results/ and docs/figures/
docker run --rm -v "$PWD:/w" -w /w/docs pandoc/latex:3.6 \
  report.md -o report.pdf --pdf-engine=xelatex
```

The 2022 report by Nasim Tavakkoli, Furkan Reha Tutaş and Melis Tuvana Sarioğlu (CS525 Data Mining, Sabancı University) is preserved unchanged in `docs/legacy/`.

10 References

1. R. M. Mohammad, F. Thabtah, L. McCluskey. *An assessment of features related to phishing websites using an automated technique*. ICITST 2012.
2. R. M. Mohammad, F. Thabtah, L. McCluskey. *Phishing Websites Features*. UCI Machine Learning Repository, id 327, 2015.
3. E. Zhu, Y. Chen, C. Ye, X. Li, F. Liu. *OFS-NN: An effective phishing websites detection model based on optimal feature selection and neural network*. IEEE Access 7, 2019.
4. M. Ahsan, R. Gomes, A. Denton. *SMOTE implementation on phishing data to enhance cybersecurity*. IEEE EIT 2018.
5. G. Ke et al. *LightGBM: A highly efficient gradient boosting decision tree*. NeurIPS 2017.
6. T. Chen, C. Guestrin. *XGBoost: A scalable tree boosting system*. KDD 2016.